

INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

Herencia y Polimorfismo

Dr. Luciano H. Tamargo
<http://cs.uns.edu.ar/~lt>
Depto. de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur, Bahía Blanca

TEMARIO

- Clases abstractas.
- Caso de estudio: Agencia publicitaria.

CLASES ABSTRACTAS

- En el diseño de una aplicación es posible definir una clase que factoriza propiedades de otras clases más específicas, **sin que existan en el problema objetos concretos** vinculados a esta clase más general.
- En este caso la clase se dice **abstracta** porque fue creada artificialmente para lograr un diseño que modele la realidad.
- En ejecución **no va a haber objetos** de software de una clase abstracta.

CLASES ABSTRACTAS

- Una clase abstracta puede incluir uno, varios, todos o ningún método abstracto.
- Si una clase hereda de una clase abstracta y no implementa todos los métodos abstractos, también debe ser definida como abstracta.
- Una clase concreta debe implementar todos los métodos abstractos de sus clases ancestro.
- El constructor de una clase abstracta sólo va a ser invocado desde los constructores de las clases derivadas.

TEMARIO

- Clases abstractas.
- Caso de estudio: Agencia publicitaria.

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

- Una **agencia publicitaria** publica avisos en diferentes medios de comunicación:
 - televisión,
 - radio,
 - diarios y
 - revistas.
- Cada **aviso** tiene asociado:
 - un nombre de fantasía,
 - un producto,
 - una empresa,
 - la fecha inicial y
 - una duración en días.

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

- Las **campañas en radios y televisión** tienen:
 - una emisora,
 - una duración en segundos y
 - una cantidad de repeticiones por día.
- No hay dos avisos de radio o TV con el mismo nombre, de una misma empresa.
- Los atributos nombre-empresa son la **clave**.

01:00
10:11
10
10
01:00
10:11
10:10
01:10
10:01
0 11
0 0
1

7

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

- Los avisos publicados en **diarios y revistas** tienen:
 - un título,
 - una cantidad de centímetros cuadrados de texto.
- No hay dos avisos impresos con el mismo nombre de una misma empresa.
- Los atributos nombre-empresa son la **clave**.

01:00
10:11
10:10
01:10
01:00
10:11
10:10
01:10
10:01
0 11
0 0
1

8

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

- El costo de una campaña en **radio o televisión** se calcula como el producto entre la cantidad de días que dura la campaña, por la cantidad de repeticiones por día, por la duración en segundos, por un monto por segundo fijo.
- El costo de una campaña en **diarios o revistas** se calcula como el producto entre la cantidad de centímetros del aviso, un monto fijo por centímetro y la cantidad de días que dura la campaña.

01:00
10:11
10:10
01:00
10:11
10:10
01:10
10:01
0 11
0 0
1

9

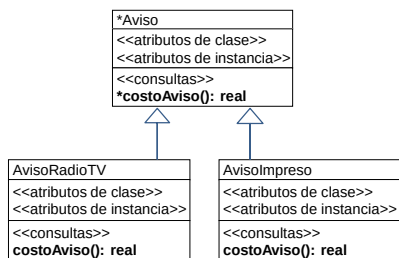
CASO DE ESTUDIO: AGENCIA PUBLICITARIA

- En el caso de estudio descrito, un aviso concreto se publica o bien en radio o en televisión o en revistas o en diarios.
- De modo que podemos definir una clase **Aviso** para factorizar atributos y comportamiento compartido.
- Esta clase no modela ningún objeto del problema real, **no tiene sentido crear objetos de software de esta clase**.
- El método **costoAviso()** es abstracto, todos los avisos tienen un costo pero la manera de calcularlo depende del medio en el que se publica.

01:00
10:11
10:10
01:10
01:00
10:11
10:10
01:10
10:01
0 11
0 0
1

10

CASO DE ESTUDIO: AGENCIA PUBLICITARIA



01:00
10:11
10:10
01:10
01:00
10:11
10:10
01:10
10:01
0 11
0 0
1

11

CASO DE ESTUDIO: AGENCIA PUBLICITARIA



01:00
10:11
10:10
01:10
01:00
10:11
10:10
01:10
10:01
0 11
0 0
1

12

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

```
abstract class Aviso{
    protected String nombre;
    protected String producto;
    protected String empresa;
    protected Fecha desde;
    protected int dias;
    ...
}
```

- El atributo **dias** indica la cantidad de días que dura la campaña.
- No hay dos avisos que coincidan en los atributos nombre-empresa, es decir, puede haber dos avisos de la misma empresa **O** con el mismo nombre, pero no con la misma empresa **Y** con el mismo nombre.

13

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

```
abstract class Aviso{
    protected String nombre;
    protected String producto;
    protected String empresa;
    protected Fecha desde;
    protected int dias;
    //Constructor
    public Aviso(String n, String p, String e, Fecha d, int di){
        ...
    }
}
```

- Como no existen instancias de una clase abstracta, el constructor de una clase no va a ser invocado explícitamente para crear objetos de la clase, sino desde los constructores de las clases derivadas.

14

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

```
class AvisoImpreso extends Aviso{
    protected static final float costoTexto = 58;
    protected String titulo;
    protected int cmTexto;

    public AvisoImpreso(String n, String p, String e, Fecha d,
        int di, String tit, int c){
        super(n,p,e,d,di);
        titulo = tit;
        cmTexto = c;
    }
    ...
}
```

15

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

```
class AvisoRadioTV extends Aviso{
    protected static final float costoSegundo = 100;
    protected String emisora;
    protected int duracion;
    protected int frecuencia;
    public AvisoRadioTV(String n, String p, String e, Fecha d,
        int di, String em, int du, int fr){
        super(n,p,e,d,di);
        emisora = em;
        duracion = du; frecuencia = fr;
    }
    ...
}
```

16

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

```
AvisoImpreso ai;
AvisoRadioTV artv;

Aviso aviso = new Aviso(...);
```

Error en compilación, la clase **Aviso** es abstracta

17

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

- La clase **Aviso** es **abstracta** porque fue creada artificialmente para factorizar los atributos y el comportamiento común a todos los avisos publicitarios.
- Podemos declarar variables de clase **Aviso** pero no crear objetos.
- El constructor de la clase **Aviso** solo va a ser invocado desde los constructores de las clases derivadas.
- En ejecución no va a haber instancias de una clase abstracta.

18

```
abstract class Aviso {
    abstract public float costoAviso();
    ...
}

class AvisoImpreso extends Aviso{
    public float costoAviso(){
        return dias * cmTexto * costoTexto;
    }
    ...
}

class AvisoRadioTV extends Aviso{
    public float costoAviso(){
        return duracion * frecuencia * costoSegundo * dias;
    }
    ...
}
```

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

- En este caso la clase **Aviso** incluye un método abstracto **costoAviso()**.
- Cada clase que especialice a la clase **Aviso** e implemente al método **costoAviso()** podrá ser una clase concreta.

```
01:00
03:11
13:10
01:10
01:00
10:11
12:10
03:10
10:1
0:11
0:0
0
```

20

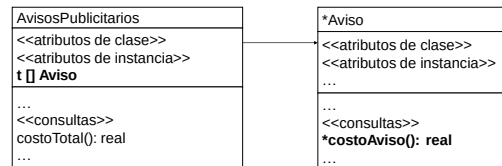
CASO DE ESTUDIO: AGENCIA PUBLICITARIA

- La clase **AvisosPublicitarios** encapsula una colección de elementos de clase **Aviso**, representada con un arreglo parcialmente ocupado y ordenado de acuerdo a la **clave**, primero por empresa y luego por nombre.
- La clase brinda servicios para:
 - Insertar ordenadamente un nuevo **Aviso**, requiere que la colección no esté llena, no exista un **Aviso** con la misma clave y el aviso no sea nulo.
 - Eliminar un **Aviso**.
 - Decidir si existe un aviso con una **clave** dada.

```
01:00
03:11
13:10
01:10
01:00
10:11
12:10
03:10
10:1
0:11
0:0
0
```

21

CASO DE ESTUDIO: AGENCIA PUBLICITARIA



```
01:00
03:11
13:10
01:10
01:00
10:11
12:10
03:10
10:1
0:11
0:0
0
```

22

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

```
class AvisosPublicitarios{
    /*Mantiene una colección de Avisos ordenados por empresa y
    luego por nombre. */
    //Atributos de instancia
    private Aviso [] t;
    private int cant;
    //Constructor
    public AvisosPublicitarios(int max){
        t = new Aviso[max];
    }
    ...
}
```

23

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

```
class Agencia{
    ...
    AvisosPublicitarios a;
    AvisoImpreso ai;
    AvisoRadioTV artv;
    a = new AvisosPublicitarios(10);
    artv = new AvisoRadioTV(...);
    ai = new AvisoImpreso(...);
    if (!a.estallena() && !a.pertenece(artv))
        a.insertar(artv);
    ...
}
```

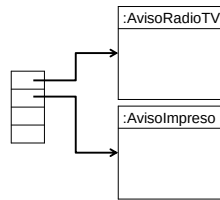
- Recorre la colección para decidir si pertenece y luego nuevamente para insertar ordenadamente.

24

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

```
class Agencia{
...
  AvisosPublicitarios a;
  AvisoImpreso ai;
  AvisoRadioTV artv;
  a = new AvisosPublicitarios(10);
  artv = new AvisoRadioTV(...);
  ai = new AvisoImpreso(...);
  if (!a.estallena() && !a.pertenece(artv))
    a.insertar(artv);
  if (!a.estallena() && !a.pertenece(ai))
    a.insertar(ai);
  float ct = a.costoTotal();
...
}
```

CASO DE ESTUDIO: AGENCIA PUBLICITARIA



Los elementos son instancias de clases derivadas de la clase Aviso

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

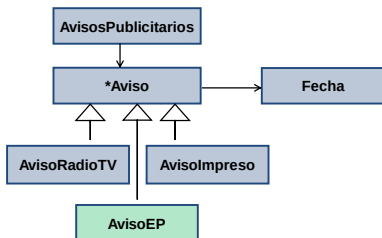
```
class AvisosPublicitarios{
...
  //Consultas
  public double costoTotal(){
    float c = 0;
    for (int i=0; i < cantAvisos(); i++)
      c = c + t[i].costoAviso();
    return c;
  }
...
}
```

- La ligadura entre el mensaje y el método `costoAviso` se establece en ejecución y depende de la clase del objeto referenciado por `t[i]`

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

- Consideremos que el diseño del problema tiene que extenderse para incluir también avisos en espacios públicos que administra la Municipalidad.
- La Municipalidad tiene codificados los espacios públicos con un código numérico que los identifica y ofrece tres tipos de carteles para colocar en esos espacios a los que denomina A, B y C.
- De modo que la clase `AvisoEP` tiene dos atributos `codigoEP` (entero) y `tipoCartel` (carácter).
- El costo de un aviso en un espacio público depende del tipo de cartel; el costo del cartel de tipo B es el doble que el de tipo A (que es una constante) y el C el doble que el B.

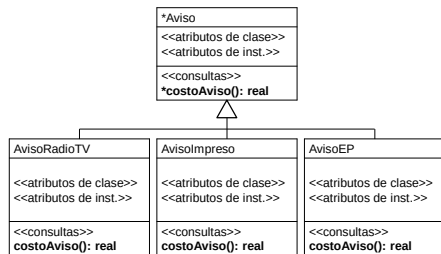
CASO DE ESTUDIO: AGENCIA PUBLICITARIA



CASO DE ESTUDIO: AGENCIA PUBLICITARIA

```
AvisoEP
<<atributos de clase>>
costoBase: real
<<atributos de instancia>>
codigoEP: entero
tipoCartel: char
<<consultas>>
costoAviso(): real
```

CASO DE ESTUDIO: AGENCIA PUBLICITARIA



01100
10011
10010
01110
01100
10011
10110
01110
1001
0 1 1
0 0
1

31

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

```

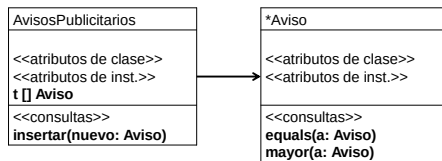
class AvisosPublicitarios{
    ...
    //Consultas
    public double costoTotal(){
        float c = 0;
        for (int i = 0; i < cantAvisos(); i++)
            c = c + t[i].costoAviso();
        return c;
    }
    ...
}
    
```

0
1
0
0
0
0
1
0
0
0

32

- El cambio en la especificación no afecta a las clases **Aviso**, **AvisoImpreso**, **AvisoRadioTV** ni **AvisosPublicitarios**.

CASO DE ESTUDIO: AGENCIA PUBLICITARIA



Para decidir la equivalencia o establecer la relación mayor se compara empresa y luego nombre

01100
10011
10010
01110
01100
10011
10010
01110
1001
0 1 1
0 0
1

33

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

- El diseñador estableció que la **clave** de un aviso es la combinación empresa-nombre.
- Dos avisos son iguales si coinciden los atributos nombre y empresa.
- Un aviso es mayor que otro si es mayor la empresa o las empresas son iguales y es mayor el nombre.

01100
10011
10010
01110
01100
10011
10010
01110
1001
0 1 1
0 0
1

34

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

Algoritmo insertar
DE nuevo
Buscar la posición de inserción
Arrastrar los Avisos
Asignar el nuevo Aviso en la posición de inserción
Incrementar la cantidad de Avisos

01100
10011
10010
01110
01100
10011
10010
01110
1001
0 1 1
0 0
1

35

- El problema es diferente a los que hemos resuelto previamente, la solución es análoga a otras propuestas antes.

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

```

public void insertar(Aviso nuevo){
    /*Requiere que la colección no esté llena y nuevo esté ligada*/
    int pos = posInsercion(nuevo);
    arrastrarDsp(pos);
    t[pos] = nuevo;
    cant++;
}
    
```

0
1
0
0
0
1
0
0
0
1
0
1
1
0
0
1

36

CASO DE ESTUDIO: AGENCIA PUBLICITARIA

```
private int posInsercion(Aviso con){
    /* Retornar la posición del primer elemento mayor a nc, o 0
    si no existe*/
    int pos = 0;
    boolean salir = false;
    while (pos < cantAvisos() && !salir){
        if (t[pos].mayor(con))
            salir = true;
        else
            pos++;
    }
    return pos;
}
```

- Mantenemos nuestro objetivo de obtener soluciones moduladas, aplicando la estrategia de dividir para conquistar en nuestros algoritmos.

37

COLECCIÓN ORDENADA: Comando auxiliar arrastrarDesp

```
private void arrastrarDesp(int pos){
    for (int i = cantAvisos(); i > pos; i--)
        t[i] = t[i-1];
}
```

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

